

6.2.1 Data Acquisition

- a) Source: PlantVillage dataset, publicly available on Kaggle ([abdallahalidev/plantvillage-dataset](https://www.kaggle.com/abdallahalidev/plantvillage-dataset)).
- b) Content: 54,303 RGB images of healthy and diseased plant leaves.
- c) Classes: 38 classes (24 diseased + 14 healthy) covering 14 crop species (apple, blueberry, cherry, corn, grape, orange, peach, pepper, potato, raspberry, soybean, squash, strawberry, tomato).
- d) Format: JPEG images with resolution varying around 256×256 pixels.

6.2.2 Data Preprocessing

Step	Description	Implementation
Resizing	All images resized to 224×224 pixels (input size required by MobileNetV2).	tf.image.resize / PIL Image.resize
Normalisation	Pixel values scaled from [0,255] to [0,1] by dividing by 255.	img_array / 255.0
Data Augmentation	Applied only to training set to improve generalisation: random rotation ($\pm 20^\circ$), width/height shift ($\pm 20\%$), horizontal flip, shear, zoom.	ImageDataGenerator with parameters
Train/Validation Split	80% training, 20% validation (stratified by class).	validation_split=0.2 in ImageDataGenerator

6.2.3 Model Selection

- a) Base Architecture: MobileNetV2 (pre-trained on ImageNet).
- b) Reason: Lightweight (≈ 3.5 million parameters), suitable for mobile/web deployment, good accuracy-speed trade-off.
- c) Custom Head: Added on top of the base model:
 - (i) Global Average Pooling 2D
 - (ii) Batch Normalisation
 - (iii) Dropout (0.3)
 - (iv) Dense (512, ReLU)
 - (v) Dropout (0.3)
 - (vi) Dense (38, softmax)

6.2.4 Training Strategy

The training follows a two-phase transfer learning approach:

Phase	Epochs	Layers trainable	Learning rate	Purpose
Phase 1	10	Only custom head (base frozen)	0.001	Learn dataset-specific features from the pre-trained representations.
Phase 2	10	Unfreeze last 30 layers of base model	0.00001	Fine-tune higher-level features for the 38-class task.

Callbacks used:

- ModelCheckpoint: saves best model based on validation accuracy.
- EarlyStopping: stops training if validation loss does not improve for 5 epochs.
- ReduceLROnPlateau: reduces learning rate by factor 0.5 when validation loss **plateaus**.

6.2.5 Evaluation Metrics

Metric	Formula	Target
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	$\geq 90\%$
Precision (per class)	$TP / (TP + FP)$	$> 85\%$
Recall (per class)	$TP / (TP + FN)$	$> 85\%$
F1-Score	$2 * (Precision * Recall) / (Precision + Recall)$	$> 85\%$
Confusion Matrix	Visual representation of misclassifications	Analysed for most confused classes

After training, the model is evaluated on the withheld validation set (20% of data). A classification report and confusion matrix are generated to identify weak classes.

6.2.6 Deployment / Inference Pipeline

Once the model is trained and saved as models/crop_disease_model.h5, it is integrated into the Flask backend as follows:

1. Model Loading (at server startup):
DiseasePredictor singleton loads the .h5 file and class names (class_names.txt) into memory.
2. Preprocessing (per request):
The uploaded image is opened with PIL, converted to RGB, resized to 224×224, converted to numpy array, normalised to [0,1], and a batch dimension is added (shape: 1,224,224,3).

3. Inference:
model.predict() returns a probability vector of length 38. The index with the highest probability is mapped to a disease name using the class list.
4. Post-processing:
 - a. Confidence score = highest probability $\times$ 100.
 - b. If confidence < 60%, the system returns “Unable to identify – please upload a clearer image”.
 - c. Otherwise, the disease name is used to query the diseases table in MySQL to retrieve organic/chemical treatments, prevention tips, and severity level.
5. History Storage:
The prediction (disease name, confidence, user ID, timestamp) is inserted into scan_history table, and the uploaded image is saved to the uploads/ folder with a UUID filename.

This section estimates the total cost for developing the **Crop Disease Detection System** using the **COCOMO II** model for effort, then converting to a 4-person team over 3 months with realistic salaries, laptop depreciation, and overheads. All figures are in **Nepali Rupees (NPR)** and the final budget falls within rs **2–4 lakhs NPR**. **Final Cost Estimation – COCOMO II Based (4 Persons, 3 Months)**

- Team size: **4 persons**
- Duration: **3 months** (12 person-months)
- Total laptop cost: RS **,3,40,000**
- Target total: $\approx$ **RS ,3,97,000**

1. Project Size (Source Lines of Code)

Component	Estimated SLOC
Flask Backend (Python)	2,500

Web Frontend (HTML/CSS/JS)	1,800
Database Schema & SQL	300
ML Training Script	400
Total	5,000 SLOC (5 KSLOC)

2. COCOMO II Early Design – Effort Estimation

$$\text{Nominal Person-Months} = 2.94 \times (5.0)^{1.10} = 2.94 \times 5.87 \approx 17.26$$

Effort Adjustment Factors (EAF):

- Precedentedness: 1.12
- Development flexibility: 0.85
- Architecture/risk: 1.00
- Team cohesion: 0.85
- Process maturity: 1.00

Product: $1.12 \times 0.85 \times 0.85 = 0.8092$

$$\text{Adjusted Person-Months} = 17.26 \times 0.8092 \approx 14.0$$

The theoretical COCOMO effort is 14 person-months; planned effort is 12 person-months (4 persons × 3 months).

3. Salary Calculation (4 persons, 3 months)

Role	Monthly Salary (NPR)	3-Month Total (NPR)
Project Lead / Full-stack	35,000	1,05,000
Backend & ML Engineer	30,000	90,000

Frontend (Web)	25,000	75,000
Database & Testing	20,000	60,000
Total	1,10,000 per month	3,30,000

4. Laptop Depreciation (3 months)

- Total laptop cost (4 laptops): ₹3,40,000
- Useful life: 36 months
- Depreciation per month: $3,40,000/36=9,444.44$, $3,40,000/36=9,444.44$
- **Depreciation for 3 months:** $9,444.44 \times 3=28,333$, $9,444.44 \times 3=28,333$ (rounded)

5. Infrastructure & Overheads (3 months)

Item	Cost (NPR)
Cloud / hosting (testing)	5,000
Internet & electricity	4,000
Miscellaneous (software, stationery)	3,000
Total	12,000

6. Contingency (Adjusted to achieve rs 3.97 lakhs)

- Basis (salaries + depreciation) = $3,30,000+28,333$
- $=3,58,333$, $3,30,000+28,333=3,58,333$
- Required contingency = $3,97,000-(3,58,333+12,000)$
 $=3,97,000-3,70,333$

$$=26,6673,97,000-(3,58,333+12,000)$$

$$=3,97,000-3,70,333=$$

$$26,667$$

- Contingency percentage $=26,667/3,58,333$

$=0.744$ use **7.5%** for simplicity.

$$\text{Contingency (7.5\%)} = 3,58,333 \times 0.075 = 26,875$$

7. Total Development Cost

Cost Component	Amount (NPR)
Salaries (4 × 3 months)	3,30,000
Laptop depreciation	28,333
Infrastructure & overheads	12,000
Contingency (7.5%)	26,875
Grand Total	3,97,208

$\approx$ **rs3,97,000 NPR** (well within the ₹2–4 lakh range).

Revised DFD Level 1 – Crop Disease Detection System

Based on your database schema and project requirements, here is the **revised Data Flow Diagram Level 1**. It decomposes the main system into **five core processes**, shows data stores (tables), and the flows between external entities (User, Admin) and internal processes.

External Entities (same as Level 0)

- **User** (Farmer)
- **Admin**

Processes

ID	Process Name	Description
1.0	Authentication	Handles user registration, login, JWT token generation, password change, and OTP-based password reset.
2.0	Disease Management	Provides disease library (list, search by crop/severity), disease details with treatments.
3.0	Prediction	Accepts leaf image, preprocesses, invokes ML model, returns predicted disease + confidence, stores scan history.
4.0	Admin Management	Enables admin to view system stats, list all users, view user details, toggle user status, change roles, and view logs.
5.0	History & Favourites	Manages user's scan history (view past predictions) and favourite diseases (if feature kept).

Note: Favourites (user_favorites table) is optional; if removed, process 5.0 handles only history.

Data Stores (Tables)

ID	Data Store	Corresponding Table
D1	Users	users
D2	Diseases	diseases
D3	Scan History	scan_history
D4	Medicines	medicines + disease_medicine_mapping (not directly accessed by user)
D5	Admin Logs	admin_logs
D6	System Settings	system_settings
D7	Password Resets	password_resets

For simplicity in DFD, we can group D4, D5, D6, D7 as part of the processes that use them.

Data Flows (Table Format)

From	To	Data Flow (Label)	Description
User →	1.0	registration details, login credentials, OTP	Input for auth
1.0 →	User	JWT token, verification status, password reset confirmation	Output
1.0 ↔	D1	user data (read/write)	Create, update users

User →	2.0	disease query (crop/severity)	Request disease list
2.0 →	User	disease list, treatment details	Output
2.0 ↔	D2	read diseases	Read diseases table
User →	3.0	leaf image (upload)	Input for prediction
3.0 →	User	disease name, confidence, treatment, severity	Output
3.0 ↔	D2	read disease treatment by name	Fetch treatment
3.0 →	D3	write scan record	Store prediction
User →	5.0	history request, add/remove favourite	Input
5.0 →	User	history list, favourites list	Output
5.0 ↔	D3	read scan_history by user_id	Retrieve history
Admin →	4.0	admin commands (view stats, user list, toggle status, change role, view logs)	Input
4.0 →	Admin	stats, user lists, user details, logs	Output
4.0 ↔	D1	read all users, update is_active, role	User management
4.0 ↔	D3	read scan counts per user	For statistics
4.0 →	D5	write admin action logs	Logging
4.0 ↔	D6	read/write system settings	Optional

How the Model Prediction Connects to the diseases Table (Treatment Flow)

You are confused about how the system links a predicted disease to the treatment information stored in the diseases table. Let me explain the **simple but effective** way your schema already supports this.

1. Model Output

The deep learning model (MobileNetV2) outputs a **disease name** as a string, for example: "Tomato___Late_blight" (or a cleaned version "Tomato Late Blight").

2. Query the diseases Table

After the model predicts the disease name, your backend code (Flask) does a **lookup** in the diseases table using that name:

sql

```
SELECT    organic_treatment,    chemical_treatment,    prevention_tips,    severity_level
FROM                                             diseases
WHERE disease_name = 'Tomato Late Blight';
```

The diseases table has a **unique** disease_name **column** (you marked it as PK in your diagram). That is the **connection point**.

3. No Foreign Key Needed

You do **not** need a foreign key from scan_history to diseases.

- scan_history stores the disease_name as a string (for historical record).
- Even if the diseases table changes later, the history keeps the exact name at the time of prediction.

- For fetching treatment, you simply query diseases by the predicted name – it's a **runtime match**, not a permanent link.

This is called **soft-referencing** and is common in medical/agricultural systems where disease names are stable identifiers.

4. Step-by-Step Flow (Your System)

text

User uploads leaf image



Model predicts: "Tomato Late Blight"



Backend queries: `SELECT * FROM diseases WHERE disease_name = "Tomato Late Blight"`



Gets: `organic_treatment, chemical_treatment, prevention_tips, severity_level`



Returns treatment to user + saves `scan_history` with `disease_name` and `confidence`

5. Your Database Schema – Small Corrections

Your diagram shows some typos that won't affect logic but could cause confusion:

Your column name	Suggested correction
<code>confidence_detected(3.2)</code>	<code>confidence DECIMAL(5,2)</code>
<code>severity_vaccine(20)</code>	<code>severity VARCHAR(20)</code>
<code>verification_type varchar(5)</code>	<code>verification_otp VARCHAR(6)</code>
<code>opt_empire_at</code>	<code>otp_expires_at</code>
<code>opt_lat_settlement</code>	<code>otp_last_sent</code>
<code>used_by user_id(1)</code>	<code>used TINYINT(1) (not a foreign key)</code>

in organic_treatment(1)	is_organic TINYINT(1)
-------------------------	-----------------------

But these do **not** break the connection between prediction and treatment.

6. If You Want a Stronger Link (Optional)

You could add a foreign key from scan_history to diseases.id, but that would require storing a numeric disease_id in scan_history. It's unnecessary for your use case because:

- Disease names are unique and rarely change.
- The extra join would add negligible performance gain.

Summary

- **Connection:** Model outputs disease name → backend uses that name to query diseases table → retrieves treatment.
- **No foreign key required** – the disease_name string works as the link.
- Your current schema already supports this completely.
- The user_favorites table (if present) is for user-saved favourites only – not involved in prediction or treatment.